Bonded knots

Katarzyna Gozdek

21.04.2026

Abstract

The main objective of this paper is to present a systematic classification of uncolored bonded knots with singularity number at most nine. Bonded knots can model proteins that contain strong bonds between distant amino acids, such as ionic bonds or disulfide bridges. We describe the methods used to achieve this classification, including the generation of bonded knot diagrams and the distinction of topological knotted types through Yamada polynomial and Reidemeister moves.

1 Introduction

Proteins perform many functions essential to life processes, including signal transduction, gene transcription, and catalysis of chemical reactions. Among these, catalytic activity is particularly prominent when it comes to knotted proteins - rare biomolecules whose backbones entangle themselves in a knot. Notably, roughly 80% of knotted proteins are enzymes [Source], while others have been identified as membrane proteins and secondary active transporters[source]. Despite ongoing research, their functions and folding mechanisms are still not well understood.

Since protein function is directly related to its conformation, an analysis of their structure may prove helpful in answering these questions. Bonded knots can be applied to model such structures. This approach captures not only the spatial knotting of the backbone but also incorporates the chemical intra-chain bonds, representing disulfide bridges, hydrogen bonds, and salt bridges. These additional constraints are represented by embedded arcs, which distinguish bonded knots from classical knots. More generally, collections of multiple chains connected by additional intra- or intermolecular bonds can be modeled as bonded links. Since protein backbones are not closed curves, a suitable closure must be introduced to enable the application of knot-theoretic methods.

Notably, this approach extends beyond knotted proteins and applies to a broader class of biomolecules containing intra-chain or intermolecular bonds.

For instance, proteins with disulfide bonds alone constitute roughly 30% of all proteins [Source], highlighting the potential relevance of bonded knots in structural biology.

As of now, bonded structures have been classified up to 7 singularities (number of crossings and vertices) [source]. In this paper, we extend this classification to 9 singularities and describe the methodology used to obtain these results.

2 Defintions

[source: https://arxiv.org/pdf/2603.18768;]

A *graph* is a collection of edges and vertices such that each edge has a pair of vertices as its endpoints. A *spatial graph* is a graph embedded in 3-space.

(pisac o 3-valent??)

[https://arxiv.org/pdf/1910.04398 ¡- dla colored, ale skopiowalam]

A *bonded knot* is a tuple (L, b), where

- L is a knot embedded in the three-sphere S^3 (or R^3),
- $b = \{b_1, b_2, \ldots, b_n\}$ is a collection of pairwise disjoint bonds (closed intervals) properly embedded in $S^3 \setminus L$.

Bonded knots can be represented as 3-valent graphs equipped with a perfect matching.

A *bonded link* is defined analogously, where L is a link. The bonds may connect different components of the link or the same component.

2.1 Projections, Diagram Equivalence and Reidemeister Moves

[source: https://arxiv.org/pdf/1912.09353]

A *knot diagram* is a regular projection of a knot onto a plane, with crossings equipped with over–under information.

The *Planar Diagram (PD) code* of a bonded knot is a combinatorial encoding of a diagram obtained by labeling its arcs and recording, for each node (crossing or bond endpoint), the incident arcs in counterclockwise order. In the case of a crossing, the ordering is initiated at an undercrossing arc. The resulting PD code is an ordered list of such node entries that completely determines the planar diagram.

Two knot diagrams are *equivalent* if there exists a finite sequence of Reidemeister moves transforming one diagram into the other.

[wiecej o tym V(?) https://arxiv.org/pdf/2603.18768]

Reidemeister moves are local transformations of knot diagrams that preserve knot type. There are three classical Reidemeister moves (I–III), together with additional moves IV and V for bonded knots (or knotted graphs), which describe interactions at vertices where bonds meet the knot.

The *singularity number* is defined as

$$s(K) = c(K) + 2b(K),$$

where $c(K)$ denotes the minimal number of crossings among all diagrams of a bonded knot K under generalized Reidemeister moves, and $b(K)$ is the number of bonds in K.

2.2 Diagram Terminology

A *vertex* is a bond endpoint. A vertex is represented in PD-code as $V[a_1, a_2, a_3]$, where the entries denote the incident arcs listed in counterclockwise order.

A *crossing* is a point where one strand passes over another. In the PD-code representation, the incident arcs are ordered counterclockwise, starting at an undercrossing arc; undercrossing arcs occupy odd positions and overcrossing arcs even positions. A crossing is denoted by $X[a_1, a_2, a_3, a_4]$. .[https://arxiv.org/pdf/1912.09353]

A *node* is either a vertex or a crossing.

An *arc* is a smooth curve joining a pair of adjacent nodes, with no vertices or crossings in its interior.

A *strand* is a simple path in the diagram whose endpoints are nodes, and whose interior contains no vertices. Crossings may occur along a strand.

An *edge* is a strand between two vertices.

A *diagram reduction* occurs when a diagram with n nodes can be transformed, via Reidemeister moves, into an equivalent diagram containing strictly fewer than n nodes.

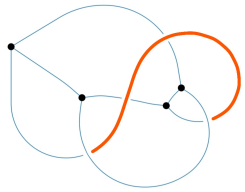
3 Methodology

3.1 Detection of Diagram-Reducing Strand Flip Move

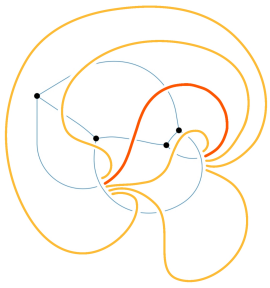
3.1.1 Idea

When a strand crosses exclusively over all other strands it encounters in the diagram (Fig. 1), it is possible to perform a *strand flip* - move the strand to an alternative position in the diagram in such a way that all of its current crossings disappear. This operation may introduce new crossings over other strands, depending on the chosen placement (Fig. 2a). Consequently, if the alternative position induces fewer crossings than the original, then the total number of crossings in a diagram decreases and the diagram is therefore reduced (Fig. 2b, 2c).

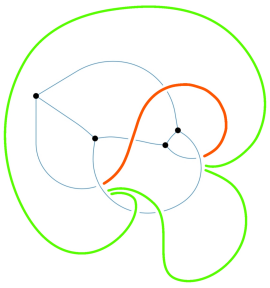
The idea above is described for strands that pass exclusively over other strands they encounter. By symmetry, the same reasoning holds for strands that pass exclusively under all encountered crossings.



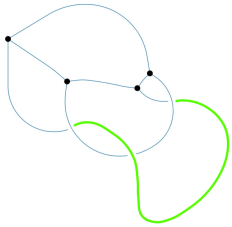
Rysunek 1: over-over path example



(a) Alternative path placement example



(b) better paths example



(c) Diagram after reduction

Rysunek 2: Caption

3.1.2 Detection Algorithm

For simplicity, the following algorithm description focuses solely on strands that pass exclusively over crossings and is analogous to the case of strands passing exclusively under crossings.

For every pair of nodes in a diagram (vertices and crossings):

1. If paths passing exclusively over crossings exist, find the one containing the largest number of crossings.
2. Compute all possible paths between the nodes and find the one that contains the smallest number of crossings.
3. If the number of crossings in the minimal-crossing path (2) is strictly smaller than in the maximal over-type path (1), a diagram reducing strand flip move is possible.

Finding an Over-Type Path

Finding an 'over'-type path between two nodes is performed by following each strand incident to the starting node and tracing it through the diagram.

The process terminates either unsuccessfully, if a vertex or an 'under' crossing is encountered, or successfully if the target node is reached. In the latter case, the traced path is the desired 'over'-type path.

Finding a Minimal-Crossing Path

Computing an alternative placement for a strand, together with its cost, which represents the number of crossings that would be created by inserting a strand along this path, can be obtained by:

1. Following an arc until a crossing or a vertex is reached
2. At a vertex, the two remaining arcs must be considered. These correspond to continuing the traversal with either a left or a right turn.
3. At a crossing, three arcs may be followed: one to the left, one to the right and one passing straight through the crossing.

The side of the path along which the strand is inserted determines whether a crossing is created. Taking this into account, the following cost model is defined:

- Turns at vertices or crossings have cost 1 if the side changes and cost 0 otherwise.
- Passing straight through a crossing preserves the side and has cost 1.

The minimal-crossing path is then obtained by selecting the path with the minimal total cost according to this cost model.